一、问题分析

本实验旨在解决病毒感染检测的问题，通过模拟病毒 DNA 序列（环状）在人类 DNA 序列（线性）中是否出现来判断感染情况。由于病毒的 DNA 是环状结构，这意味着病毒 DNA 序列可以以任意起点开始匹配，这增加了匹配算法的复杂度。因此，需要对传统的字符串模式匹配算法进行改进，以适应环状模式串的搜索需求。

二、总体设计

为了提高检测效率，选择 KMP（Knuth-Morris-Pratt）算法作为核心匹配算法。KMP 算法能够在线性时间内完成模式匹配，避免了 BF（Brute Force）算法可能产生的大量冗余比较。考虑到病毒 DNA 的环状特性，我们将病毒 DNA 序列的所有旋转形式都视为潜在的匹配模式，并逐一尝试与人的 DNA 序列进行匹配。

三、详细设计

1. **KMP 算法实现**
    - **LPS 数组计算：**`computeLPSArray` 函数用于构建最长前缀后缀（LPS, Longest Prefix Suffix）数组。该数组记录了模式串中每个位置的最长公共前后缀长度，有助于在遇到不匹配时快速回溯到正确的位置继续匹配。
    - **模式匹配过程：**`kmpSearch` 函数实现了 KMP 算法的核心逻辑。它接受文本串和模式串作为参数，利用预先计算好的 LPS 数组指导匹配过程。如果找到完全匹配，则返回匹配结束位置；否则返回 0 表示未找到。
2. **环状模式串处理**
    - 鉴于病毒 DNA 的环状性质，我们需要考虑所有可能的旋转形式。通过循环地将病毒 DNA 序列的第一个字符移动到最后，生

成其所有的旋转变体，并用 KMP 算法逐个检查这些变体是否存在于人的 DNA 序列中。

3. **主程序流程**
   - 主程序采用循环读取输入的方式处理多组测试数据。对于每组数据，首先调用 `kmpSearch` 尝试直接匹配；如果没有找到匹配，则继续生成并测试病毒 DNA 序列的所有旋转形式。一旦找到匹配或遍历完所有旋转形式，输出结果（“YES”或“NO”），然后读取下一组数据，直到遇到结束标记（"0 0"）。
4. **优化点**
   - 在实际应用中，我们可以进一步优化代码性能。例如，可以通过预处理病毒 DNA 序列，只生成那些有可能出现在人 DNA 中的旋转形式，减少不必要的匹配尝试。此外，考虑到实际病毒 DNA 序列的长度通常较长，可以探索使用更高效的多模式匹配算法或者并行化技术来加速匹配过程。

四、调试分析

在开发过程中，遇到了几个主要问题：

- 初期版本中，未能正确处理病毒 DNA 序列的环状特性，导致部分测试用例失败。通过引入旋转机制解决了这个问题。
- KMP 算法的实现初期存在边界条件处理不当的问题，经过多次调试和测试，最终确保了算法的正确性。
- 对于非常长的 DNA 序列，程序运行时间过长。为此，我们优化了代码结构，并考虑了上述提到的优化方案。

五、用户手册

使用说明

1. 运行程序：执行编译后的可执行文件，并按照提示输入病毒和人的 DNA 序列。每次输入两个字符串，分别代表病毒 DNA 和人 DNA 序列，之间以空格分隔。输入"0 0"结束程序。

2. 查看结果：程序会根据输入自动判断是否存在匹配，并输出“YES”或“NO”。

注意事项

- 输入必须严格按照格式要求，即两个由小写字母组成的字符串，之间以单个空格分隔。
- 输入结束后，请务必输入"0 0"以正确终止程序。

六、测试结果

进行了广泛的测试，包括但不限于以下几类：

- 正常情况：病毒 DNA 序列确实存在于人 DNA 序列中，程序输出“YES”。
- 病毒不存在：当给定的人 DNA 序列中不包含任何病毒 DNA 序列的变体时，程序输出“NO”。
- 边界条件：测试了各种极端情况，比如非常短或非常长的 DNA 序列，以及病毒 DNA 序列恰好位于人 DNA 序列两端的情况。
- 环状特性：验证了程序能否正确处理病毒 DNA 序列的环状特性，即病毒 DNA 序列的各个旋转形式都能被正确识别。

所有测试均得到预期的结果，证明了程序的正确性和鲁棒性。

附录-程序

```
// main.cpp 文件

#include <iostream>
#include <string>
#include <vector>
#include <list>
#include <set>
#include <map>
#include <queue>
#include <algorithm>

using namespace std;

void computeLPSArray(const std::string& pattern, std::vector<int>& lps) {
	int m = pattern.length();
	int len = 0;
	lps[0] = 0;

	int i = 1;
	while (i < m) {
		if (pattern[i] == pattern[len]) {
			lps[i] = ++len;
			++i;
		}
		else {
			if (len != 0)
				len = lps[len - 1];
			else {
				lps[i] = 0;
				++i;
			}
		}
	}
}

int kmpSearch(const std::string& text, const std::string& pattern) {
	int n = text.length();
	int m = pattern.length();
	std::vector<int> lps(m, 0);
	computeLPSArray(pattern, lps);
```

```
    int i = 0; //  文本索引
    int j = 0; //  模式索引

    while (i < n) {
        if (pattern[j] == text[i]) {
            ++i;
            ++j;
        }

        if (j == m) {
            return i; //  找到匹配
        }
        else if (i < n && pattern[j] != text[i]) {
            if (j != 0)
                j = lps[j - 1];
            else {
                ++i;
            }
        }
    }
    return 0; //  未找到匹配
}

int main()
{
    string text,pattern;
    while(true) {
        cin>>pattern>>text;
        if(pattern=="0"&&text=="0")
            break;
        int n=pattern.size();
        int flag;
        while(n--) {
            flag = kmpSearch(text, pattern);
            if(flag)
                break;
            //将第一个字母移动到末尾，环状结构模拟
            rotate(pattern.begin(), pattern.begin() + 1, pattern.end());
        }
        if (flag)
            cout << "YES" << endl;
        else
            cout << "NO" << endl;
    }
```

```
    return 0;
}
```